

■ ****ProductController.java Explanation (Tanglish)****

****Package Declaration****

package com.inventory.controller;

➔■ Ithu controller package la iruka class, product related API handle panna use agum.

****Imports****

➔■ Model (Product), Repository (ProductRepository), Service (ProductService, ExcelExportService) import panniruka.

➔■ Spring Boot annotations, ResponseEntity, HttpHeaders etc. use panniruka.

➔■ Java built-in classes like List, Map, ByteArrayInputStream, IOException use panniruka.

****Class Declaration****

@RestController

@RequestMapping("/api/products")

➔■ REST API controller for products.

➔■ API base URL: /api/products.

****Autowired Dependencies****

@Autowired ProductService productService;

@Autowired ProductRepository productRepository;

@Autowired ExcelExportService excelExportService;

➔■ Spring automatically inject pannum objects for DB and Excel operations.

■ ****1. Get All Products****

@GetMapping

public List getAllProducts()

➔■ Database la iruka ella products fetch pannum and return pannum.

■ ****2. Get Product by ID****

@GetMapping("/{id}")

➔■ PathVariable use panni product ID la based fetch pannum.

➔■ Product found ahna 200 OK, illa na 404 NOT FOUND.

■ ****3. Add New Product****

@PostMapping

➔■ Frontend la irundhu product details @RequestBody la eduthutu

➔■ productService.saveProduct(product) call pannum and CREATED (201) return pannum.

■ ****4. Update Product****

@PutMapping("/{id}")

➔■ Existing product iruka check pannum.

➔■ Irundha update pannum, illa na 404 return pannum.

■ ****5. Delete Product****

@DeleteMapping("/{id}")

➔■ Product delete pannum and 204 (No Content) return pannum.

■ ****6. Export to Excel****

@GetMapping("/export/excel")

➔■ Database la iruka products list fetch pannum.

➔■ ExcelExportService use panni Excel file ah convert pannum.

➔■ File name = products.xlsx

➔■ Browser la automatically download aagum (Content-Disposition header).

■ ****7. Place Order (Main Logic)****

@PutMapping("/order")

➔■ Input: JSON with productName, model, Unit.

Steps Inside:

1■■ Extract productName, model, orderQuantity from request.

2■■ Find product using `findByProductNameAndModel()`.

3■■ Product not found -> return 404.

4■■ Stock <= 0 -> set status OUT_OF_STOCK & return error.

5■■ Stock < orderQuantity -> insufficient stock message.

6■■ Else:

- Update stock: current - orderQuantity

- Update total price: pricePerQuantity * updatedStock

- If updatedStock == 0 → status OUT_OF_STOCK

- Save product again.

■ Success message with remaining stock & updated total price.

****Error Handling:****

Catch block -> if any exception, return 500 INTERNAL_SERVER_ERROR with message.

****Flow Summary****

1■■ User sends order request (productName + model + Unit).

2■■ Backend finds product.

3■■ Checks stock availability.

4■■ Updates DB with new quantity & total price.

5■■ Returns success/failure message.

Full controller manages CRUD, Excel export, and order placement ■